

Lecture 7

간단한 아케이드 게임
적의 동작과 각종 효과
Unity를 이용한 2D 게임프로그래밍

강영민

동명대학교 게임학부

목차

- 1 적의 슈팅
- 2 폭발 효과 – Particle System
- 3 카메라 흔들기 효과
- 4 음향 효과
- 5 전체 구조 정리
- 6 정리
- 7 도전 과제

◎ 이번 강의에서 배울 내용

- **적의 슈팅 제어** – 자동 발사, 방향 및 속도 조정
- **각종 이펙트 추가**
 - 입자 시스템 (Particle System) – 폭발 효과
 - 카메라 흔들기 효과 (Camera Shake)
 - 음향 효과 (Audio)

적의 슈팅 - Shooter.cs 재사용

이미 만들어 놓은 스크립트 활용

- 이전 강의에서 만든 **Shooter.cs**를 적에도 적용
- 모든 적 Prefab에 Shooter 컴포넌트 추가
- 적 발사체(Projectile_enemy)를 Projectile Prefab에 연결

Shooter 컴포넌트 기본 설정

속성	값
Projectile Prefab	Projectile_enemy
Projectile Speed	10
Projectile Life Time	5
Firing Rate	0.2
Is Firing	체크 해제

Shooter.cs에서 자동 발사란?

❓ 왜 자동 발사가 필요한가?

- 플레이어는 **입력(버튼)**으로 발사를 제어
- 적은 입력이 없으므로 **자동으로 발사**해야 함
- 같은 Shooter.cs를 사용하되, 자동 발사 **옵션**을 추가하여 재사용

★ 핵심 아이디어

하나의 스크립트에 autoShoot 플래그를 추가하여
플레이어용/적용 모두 대응 → **코드 재사용**의 좋은 예

적은 입력이 아니라 자동으로 발사

Shooter.cs 수정 - 자동 발사 옵션 추가

```
1 public class Shooter : MonoBehaviour
2 {
3     [SerializeField] GameObject projectilePrefab;
4     [SerializeField] float projectileSpeed = 10f;
5     [SerializeField] float projectileLifeTime = 5f;
6     [SerializeField] float firingRate = 0.2f;
7     [SerializeField] bool autoShoot = false;
8
9     public bool isFiring = false;
10    Coroutine firingRoutine;
11
12    void Start()
13    {
14        if (autoShoot)
15        {
16            isFiring = true;
17        }
18    }
19 }
```

적 Prefab 설정

적 Prefab의 Shooter 컴포넌트에서 **Auto Shoot**을 체크하면 게임 시작 시 자동 발사

⚠ 문제: 적도 위쪽으로 발사한다

- 현재 Shooter.cs의 FireContinuously()에서
`rb.linearVelocity = transform.up * projectileSpeed;`
- `transform.up`은 위쪽 방향 → 적도 위로 발사!
- 적은 **아래쪽**(플레이어 방향)으로 발사해야 함

💡 해결: autoShoot이면 반대 방향

autoShoot이 true이면 `-transform.up` 사용
→ 적의 발사체가 아래쪽으로 날아감

자동 발사인 경우 반대 방향으로 속도 지정

FireContinuously() 수정

```
1 IEnumerator FireContinuously()  
2 {  
3     while (true)  
4     {  
5         // firing action  
6         GameObject missile = Instantiate(projectilePrefab,  
7             transform.position, Quaternion.identity);  
8  
9         Rigidbody2D rb = missile.GetComponent<Rigidbody2D>();  
10        if (rb != null)  
11        {  
12            if (autoShoot)  
13            {  
14                rb.linearVelocity = -transform.up * projectileSpeed;  
15            }  
16            else  
17            {  
18                rb.linearVelocity = transform.up * projectileSpeed;  
19            }  
20        }  
21  
22        Destroy(missile, projectileLifeTime);  
23        yield return new WaitForSeconds(firingRate);  
24    }  
25 }  
26
```


적기의 발사 속도 조정

문제: 적의 발사가 너무 빠르다

플레이어와 같은 `firingRate`를 사용하면 적의 총알이 너무 많아진다.
적은 좀 더 느리게 발사하도록 조정이 필요하다.

발사 간격에 지연 배율 적용

```
1 Destroy(missile, projectileLifeTime);  
2 float delay = autoShoot ? 4f : 1f;  
3 yield return new WaitForSeconds(delay * firingRate);  
4
```

★ 핵심

`autoShoot(적)`이면 발사 간격을 **4배**로 늘려서 적절한 난이도를 유지한다.
삼항 연산자 조건 ? 참값 : 거짓값을 활용한 간결한 분기 처리

Particle System이란?

🔥 Particle System 개요

Particle System은 수많은 작은 입자(파티클)를 생성·관리하여 폭발, 연기, 불꽃, 마법 효과 등을 표현하는 Unity 컴포넌트이다.

- 각 파티클은 위치, 속도, 색상, 크기, 수명 등의 속성을 가짐
- 시간에 따라 자동으로 변화 (페이드아웃, 크기 감소 등)
- 게임의 **시각적 피드백**을 풍부하게 만드는 핵심 요소

★ 이번 강의에서의 활용

캐릭터가 파괴될 때 **폭발 이펙트**를 Particle System으로 구현한다.

☰ Particle System 제작 순서

- ❶ **Particle System 생성**: Hierarchy → Effects → Particle System
- ❷ 폭발 효과를 보일 **스프라이트 다운로드**: 강의 홈페이지에서 다운로드 후 AssetPack에 포함
- ❸ 입자 시스템에 적용할 **재질(Material)** 만들기
- ❹ Particle System에 만든 재질을 **적용**
- ❺ 각종 **설정 변경**: Texture Sheet Animation, Color over Lifetime, Shape 등
- ❻ 완성된 Particle System을 **Prefab**으로 만들기

Material 만들기

입자 시스템에 적용할 재질 생성

- 1 Asset 폴더에서 우클릭 → Create → **Material**
- 2 이름: mExplosion

재질 설정

- **Shader** 변경: Particles → Standard Unlit
- **Rendering Mode**: Additive (블렌딩 방법)
- **Map - Albedo** (색상 결정): 폭발 스프라이트 연결

Additive 블렌딩이란?

배경 색상에 파티클 색상을 **더하는** 방식. 밝은 이펙트(폭발, 빛)에 적합하다.
검정 배경에서는 투명하게 보이고, 밝은 부분이 더욱 밝게 빛난다.

Particle System에 재질 적용

Renderer 모듈에서 Material 설정

- 1 Particle System의 Inspector 열기
- 2 하단의 **Renderer** 모듈 펼치기
- 3 **Material** 항목에 만든 재질(mExplosion)을 드래그하여 연결

Renderer 모듈의 주요 속성

속성	설명
Render Mode	파티클 렌더링 방식 (Billboard 등)
Material	파티클에 사용할 재질
Sort Mode	파티클 정렬 방식
Min/Max Particle Size	화면에 보이는 최소/최대 크기

Particle System 설정 변경 – Texture Sheet Animation

Texture Sheet Animation 설정

- **Texture Sheet Animation** 모듈 활성화
- Sprite Sheet에서 한 영역만 가져오기: 스프라이트 시트를 **3개의 타일**로 변경
 - Mode: Grid
 - Tiles: X=3, Y=1

애니메이션 설정

- 파티클이 존재하는 동안 **두 번**의 사이클로 스프라이트 애니메이션 실행
- **Cycles**: 2
- **Start Frame**: 0 ~ 2 (랜덤화)
- → 각 파티클이 다른 프레임에서 시작하여 자연스러운 효과

Particle System 설정 변경 – 색상과 형태

💧 Color over Lifetime

- **Color over Lifetime** 모듈 활성화
- Gradient Editor에서 색상 변화 설정
- 시작: 노란색/주황색
- 끝: 투명하게 페이드아웃
- 투명도와 색상 모두 조정 가능

● Shape – 분출 형태

- **Shape** 모듈에서 형태 변경
- Shape: **Circle**
- 파티클이 원형으로 퍼져나감
- → 폭발 효과에 적합

★ Particle System 주요 모듈 요약

모듈	역할
Main	Duration, Start Lifetime, Start Speed, Start Size 등 기본 설정
Emission	초당 방출 수 (Rate over Time)
Shape	파티클 방출 형태 (Circle, Cone 등)
Color over Lifetime	시간에 따른 색상/투명도 변화
Size over Lifetime	시간에 따른 크기 변화
Texture Sheet Animation	스프라이트 시트 애니메이션

Particle System 세부 조정

☰ Main 모듈 세부 설정

속성	값	설명
Duration	5	파티클 시스템 지속 시간
Start Lifetime	0.1 ~ 0.5	각 파티클의 수명 (랜덤 범위)
Start Speed	3	파티클 초기 속도
Start Size	1	파티클 초기 크기
3D Start Rotation	Z: 0 ~ 360	랜덤 회전

📈 Size over Lifetime

- **Size over Lifetime** 모듈 활성화
- 커브 설정: 시작은 크게 → 시간이 지남에 따라 작아짐
- → 폭발이 점점 작아지며 사라지는 자연스러운 효과

입자 시스템을 프리팹으로

Prefab으로 저장

- 1 완성된 Particle System을 **Prefabs** 폴더에 끌어다 놓기
- 2 적절한 이름으로 변경: explosion
- 3 Hierarchy에서 원본 삭제

왜 Prefab으로 만드는가?

- 캐릭터가 파괴될 때마다 Instantiate()로 폭발 이펙트를 생성
- Prefab으로 만들어야 코드에서 [SerializeField]로 참조 가능
- 여러 캐릭터(적, 플레이어)에서 동일한 폭발 효과 재사용

Health.cs 수정 – 파괴될 때 폭발 효과

Health.cs – 폭발 이펙트 추가

```
1 public class Health : MonoBehaviour
2 {
3     [SerializeField] int health = 50;
4     [SerializeField] ParticleSystem explosion;
5
6     void TakeDamage(int damage)
7     {
8         health -= damage;
9         if (health <= 0)
10         {
11             if (explosion != null) ExplosionEffect();
12             Destroy(gameObject);
13         }
14     }
15
16     void ExplosionEffect()
17     {
18         ParticleSystem effect =
19             Instantiate(explosion, transform.position,
20                 Quaternion.identity);
21
22         Destroy(effect.gameObject, effect.main.duration);
23     }
24 }
25
```

ExplosionEffect() 동작 원리

⚙️ 코드 분석

- 1 `Instantiate(explosion, transform.position, Quaternion.identity)`
→ 현재 오브젝트 위치에 폭발 Particle System **복제본 생성**
- 2 `Destroy(effect.gameObject, effect.main.duration)`
→ 파티클 시스템의 Duration만큼 유지 후 **자동 파괴**

⚠️ 주의: 메모리 관리

`Destroy()`를 호출하지 않으면 폭발 이펙트가 씬에 계속 쌓인다!
반드시 `effect.main.duration` 후에 자동 파괴되도록 설정해야 한다.

★ `effect.main.duration`

`ParticleSystem.main`: Particle System의 Main 모듈에 접근
`.duration`: Main 모듈에서 설정한 지속 시간 값을 가져옴

카메라 흔들기(Camera Shake)란?

📺 Camera Shake의 역할

- 플레이어가 피격당했을 때 카메라를 흔들어 **충격감**을 전달
- 단순한 시각 효과지만 게임의 **몰입감**을 크게 높여줌
- 화면 전체가 잠깐 흔들리므로 플레이어에게 “맞았다”는 피드백 제공

★ 구현 전략

- 1 **ShakeCam.cs** 스크립트를 만들어 Main Camera에 추가
- 2 Health.cs에서 피격 시 **ShakeCam.Play()** 호출
- 3 코루틴으로 일정 시간 동안 카메라 위치를 랜덤하게 변경

ShakeCam.cs – 카메라에 추가

```
1 using UnityEngine;
2 using System.Collections;
3
4 public class ShakeCam : MonoBehaviour
5 {
6     [SerializeField] float shakeDuration = 1f;
7     [SerializeField] float shakeMagnitude = 0.5f;
8
9     Vector3 initialPos;
10
11     void Start()
12     {
13         initialPos = transform.position;
14     }
15
16     public void Play()
17     {
18         StartCoroutine(Shake());
19     }
20
21     IEnumerator Shake()
22     {
23         float elapsedTime = 0f;
24         while (elapsedTime < shakeDuration)
25         {
26             transform.position = initialPos +
27                 (Vector3) Random.insideUnitCircle * shakeMagnitude;
28             elapsedTime += Time.deltaTime;
29             yield return new WaitForEndOfFrame();
30         }
31     }
32 }
```

🔍 핵심 코드 해설

코드	설명
<code>initialPos</code>	카메라의 원래 위치를 저장
<code>Random.insideUnitCircle</code>	반지름 1인 원 내부의 랜덤 2D 좌표 반환
<code>* shakeMagnitude</code>	흔들림의 강도(크기) 조절
<code>WaitForEndOfFrame()</code>	현재 프레임 렌더링이 끝날 때까지 대기
<code>transform.position = initialPos</code>	흔들기가 끝나면 원래 위치로 복원

💡 설정값 가이드

- **Shake Duration:** 1초 - 흔들림 지속 시간
- **Shake Magnitude:** 0.5 - 흔들림 크기 (너무 크면 어지러움)

Health.cs에서 카메라 흔들기 실행

Health.cs – ShakeCam 연동

```
1 public class Health : MonoBehaviour
2 {
3     [SerializeField] int health = 50;
4     [SerializeField] ParticleSystem explosion;
5     [SerializeField] bool applyCamShake;
6     ShakeCam shakeCam;
7
8     void Awake()
9     {
10         shakeCam = Camera.main.GetComponent<ShakeCam>();
11     }
12
13     void TakeDamage(int damage)
14     {
15         health -= damage;
16         ShakeCamera();
17         if (health <= 0)
18         {
19             if (explosion != null) ExplosionEffect();
20             Destroy(gameObject);
21         }
22     }
23
24     void ShakeCamera()
25     {
26         if (shakeCam != null && applyCamShake)
27         {
28             Debug.Log("shaking");
29             shakeCam.Play();
30         }
31     }
32 }
```

카메라 흔들기 - 설정

⚙ 적용 방법

- **플레이어** Prefab의 Health 컴포넌트에서 Apply Cam Shake 체크
- 적 Prefab에서는 체크 **해제** (적이 맞을 때 카메라가 흔들리면 불필요)

ℹ 왜 applyCamShake 플래그를 쓰는가?

- 같은 Health.cs를 플레이어와 적 모두 사용
- 플레이어가 맞을 때만 카메라가 흔들려야 의미 있음
- [SerializeField] bool applyCamShake로 Inspector에서 선택적 적용

★ Camera.main.GetComponent<ShakeCam>()

Camera.main: 씬의 메인 카메라 참조

GetComponent<ShakeCam>(): 카메라에 붙은 ShakeCam 컴포넌트를 가져옴

음향 효과란?

🔊 게임에서 음향의 중요성

- 시각 효과와 함께 **청각 피드백**을 제공하여 몰입감 향상
- 발사, 피격, 폭발 등 핵심 액션에 사운드를 연결
- Unity의 **AudioSource**와 **AudioClip**을 활용

📁 애셋 준비

- AssetPack → Bonus 폴더에 사운드 파일 준비
- sfx_laser1, sfx_laser2: 레이저 발사음
- sfx_lose: 패배 사운드
- sfx_shieldUp, sfx_shieldDown: 쉴드 효과음
- sfx_twoTone: 알림음
- sfx_zap: 전기 효과음

Unity Audio 시스템 개요

AudioClip vs AudioSource vs AudioListener

컴포넌트	역할
AudioClip	사운드 파일 자체 (WAV, MP3 등)
AudioSource	AudioClip을 재생하는 컴포넌트 (스피커 역할)
AudioListener	소리를 듣는 컴포넌트 (마이크 역할, 보통 카메라에 부착)

★ AudioSource.PlayClipAtPoint()

`AudioSource.PlayClipAtPoint(clip, position, volume)`

- 지정한 위치에 임시 AudioSource를 생성하여 클립을 **한 번** 재생
- 재생이 끝나면 자동 파괴 - 별도의 AudioSource 컴포넌트 불필요

AudioPlayer.cs – 음향 재생 관리

```
1 using UnityEngine;
2
3 public class AudioPlayer : MonoBehaviour
4 {
5     [SerializeField] AudioClip shootingClip;
6     [SerializeField] [Range(0f, 1f)] float shootingVolume = 1f;
7
8     public void PlayShootingClip()
9     {
10         if (shootingClip != null)
11         {
12             AudioSource.PlayClipAtPoint(
13                 shootingClip,
14                 Camera.main.transform.position,
15                 shootingVolume
16             );
17         }
18     }
19 }
20
```

AudioPlayer 설정

⚙ Hierarchy에 AudioPlayer 오브젝트 생성

- 1 Hierarchy에서 **Create Empty** → 이름: AudioPlayer
- 2 **AudioPlayer.cs** 스크립트 연결
- 3 Inspector에서 **Shooting Clip**에 사운드 파일 연결
- 4 **Shooting Volume: 1** (0~1 범위)

💡 [Range(0f, 1f)] 속성

Inspector에서 **슬라이더**로 표시되어 0과 1 사이의 값만 입력할 수 있다.
볼륨처럼 범위가 정해진 값에 사용하면 편리하다.

📄 Camera.main.transform.position

AudioListener가 카메라에 있으므로, 소리를 카메라 위치에서 재생하면 거리에 따른 감쇠 없이 일정한 볼륨으로 들린다.

Shooter.cs에서 음향 재생

Shooter.cs – AudioPlayer 연동

```
1 public class Shooter : MonoBehaviour
2 {
3     [SerializeField] GameObject projectilePrefab;
4     [SerializeField] float projectileSpeed = 10f;
5     [SerializeField] float projectileLifeTime = 5f;
6     [SerializeField] float firingRate = 0.2f;
7     [SerializeField] bool autoShoot = false;
8
9     public bool isFiring = false;
10    Coroutine firingRoutine;
11
12    AudioPlayer audioPlayer;
13
14    void Awake()
15    {
16        audioPlayer = FindAnyObjectByType<AudioPlayer>();
17    }
18 }
```

Shooter.cs – FireContinuously에서 사운드 재생

FireContinuously() – 발사 시 음향 추가

```
1 IEnumerator FireContinuously()
2 {
3     while (true)
4     {
5         if (audioPlayer != null)
6         {
7             audioPlayer.PlayShootingClip();
8         }
9         // firing action
10        GameObject missile = Instantiate(projectilePrefab,
11            transform.position, Quaternion.identity);
12
13        Rigidbody2D rb = missile.GetComponent<Rigidbody2D>();
14        if (rb != null) ...
15
16        Destroy(missile, projectileLifeTime);
17        float delay = autoShoot ? 4f : 1f;
18        yield return new WaitForSeconds(delay * firingRate);
19    }
20 }
21
```

★ FindAnyObjectByType<T>()

씨에서 AudioClip 타입의 오브젝트를 검색하여 참조를 얻는다

Shooter.cs – 최종 전체 구조

🧩 Shooter.cs에 추가된 기능 정리

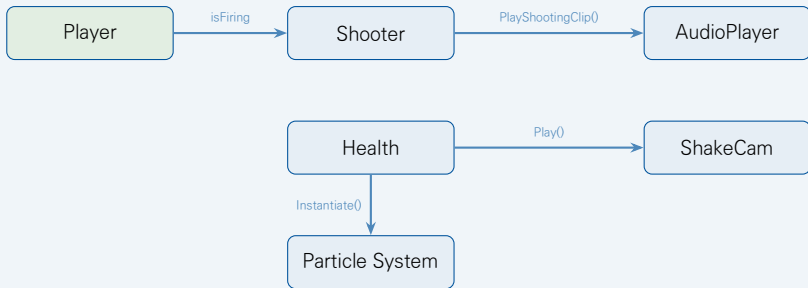
추가 기능	설명
autoShoot	적의 자동 발사 on/off
방향 반전	autoShoot이면 -transform.up으로 아래 발사
발사 속도 조정	autoShoot이면 firingRate * 4f로 느리게
음향 재생	AudioPlayer.PlayShootingClip() 호출

★ 확장된 Health.cs 구조

추가 기능	설명
explosion	파괴 시 폭발 Particle System 생성
applyCamShake	플레이어 전용 카메라 흔들기 on/off
ShakeCamera()	ShakeCam.Play() 호출

컴포넌트 간 관계도

스크립트 간 상호작용



📖 이번 강의 요약

- ❶ **적의 슈팅**: autoShoot 플래그로 자동 발사, 방향 반전, 속도 조정
- ❷ **Particle System**: Material 생성 → 설정 변경 → Prefab으로 저장
- ❸ **폭발 효과**: Health.cs에서 파괴 시 Particle System Instantiate
- ❹ **카메라 흔들기**: ShakeCam.cs – 코루틴으로 랜덤 위치 이동
- ❺ **음향 효과**: AudioPlayer.cs + PlayClipAtPoint()로 발사음 재생

💡 게임 완성도를 높이는 3요소

- **시각 효과** – Particle System (폭발, 연기)
- **카메라 효과** – Camera Shake (충격감)
- **청각 효과** – Sound Effect (피드백)

이 세 가지를 조합하면 “느낌 있는 게임”이 된다!

도전 과제 1

플레이어가 미사일을 맞아 데미지를 입을 때에
사용할 입자 시스템을 추가해 보라

힌트

- 폭발과 다른 색상/크기의 Particle System을 새로 만들어 보자
- Health.cs의 `TakeDamage()`에서 체력이 0 이하가 아닐 때도 이펙트를 재생

도전 과제 2

적의 발사와 플레이어 발사가
서로 다른 사운드가 나오도록 해 보라

힌트

- AudioPlayer.cs에 적용 발사 클립을 추가 필드로 만들어 보자
- Shooter.cs에서 `autoShoot` 여부에 따라 다른 메서드를 호출

?

이제 게임 근처에 도달?

감사합니다